

Programming Paradigms

Building Microservice Systems

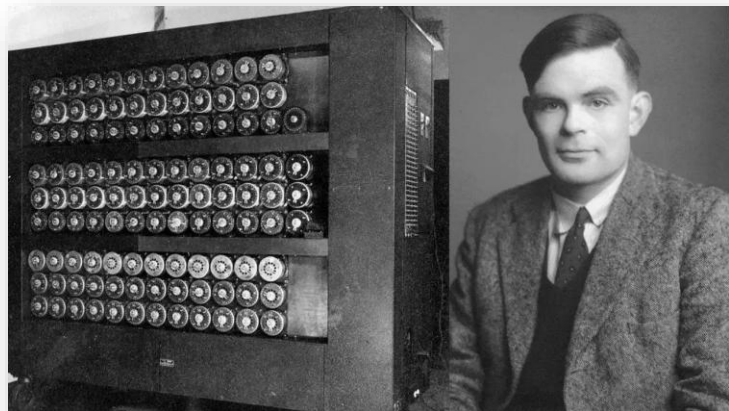
Ali Abrishami

Seyyed Hossein Mirhosseini



Programming History

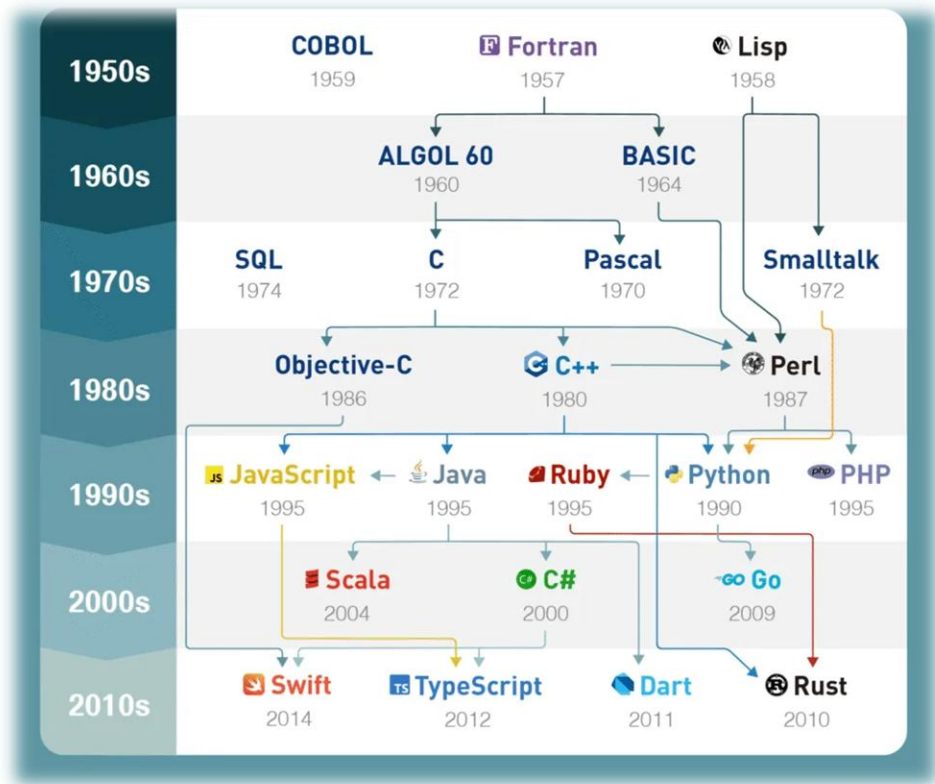
- Software architecture begins with the code.
- We will begin our discussion of architecture by looking at what we've learned about code.
- In 1938, Alan Turing laid the foundations of what was to become computer programming.
- He was the first to understand that:
 - programs were simply **data**.



Programming History (cont.)

- First, in the late 1940s, came assemblers.
These “languages” relieved the programmers of the drudgery of translating their programs into binary.
- In 1951, Grace Hopper invented A0, the first compiler.
- Fortran was invented in 1953.
- What followed was an unceasing flood of new programming languages:
 - COBOL, PL/1, SNOBOL, C, Pascal, C++, Java, ad infinitum.

Programming History (cont.)



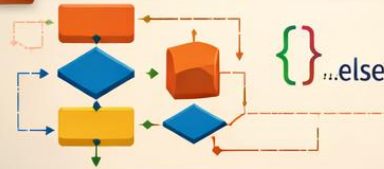
Paradigm Overview

Programming Paradigms

- Another, probably more significant, revolution was in **programming paradigms**.
- Paradigms are **ways of programming**, relatively unrelated to languages.
- A paradigm tells you **which** programming structures to use, and **when** to use them.
- To date, there have been three such paradigms:
 - Structured Programming
 - Object-Oriented Programming
 - Functional Programming

Programming Paradigms

Structured Programming



Sequence
Selection
Iteration

Object-Oriented Programming



Classes
Objects
Inheritance

Programming Paradigms

Functional Programming

$\lambda \rightarrow f(x) \rightarrow \text{result}$

Pure Functions
• Higher-Order Functions •
Immutability

Structured Programming

- The first paradigm to be adopted (but not the first to be invented) was structured programming, which was discovered by ***Edsger Wybe Dijkstra*** in 1968.
- Dijkstra showed that the use of unrestrained jumps (goto statements) is **harmful** to program structure.
- He replaced those jumps with the more familiar if/then/else and do/while/until constructs.
- Structured programming imposes discipline on **direct transfer of control**.

Object-Oriented Programming

- The programmers noticed that the function call stack frame in the ALGOL language could be moved to a **heap**, thereby allowing local variables declared by a function to exist long after the function returned.
- The function became a **constructor** for a class, the local variables became **instance variables**, and the nested functions became **methods**.
- This led inevitably to the discovery of polymorphism.
- Object-oriented programming imposes discipline on **indirect transfer of control**.

Functional Programming

- The third paradigm, which has only recently begun to be adopted, was the first to be invented.
- Functional programming is the direct result of the work of Alonzo Church, who in 1936 invented **λ-calculus**.
- A foundational notion of λ-calculus is **immutability**—that is, the notion that the values of symbols **do not change**.
- This effectively means that a functional language has **no assignment statement**.
- Functional programming imposes discipline upon **assignment**.

Comparing Paradigms

- Each of the paradigms **removes** capabilities from the programmer.
- None of them adds new capabilities.
- Each imposes some kind of extra discipline that is negative in its intent.
- The paradigms tell us **what not to do**, more than they tell us **what to do**.
- The three paradigms together remove goto statements, function pointers, and assignment.

Paradigms Usages

- Further evidence that there are no more such paradigms is that they were all discovered within the ten years between 1958 and 1968.
- We use **polymorphism** as the mechanism to cross architectural boundaries;
- We use **functional programming** to impose discipline on the location of and access to data;
- And, we use **structured programming** as the algorithmic foundation of our modules.

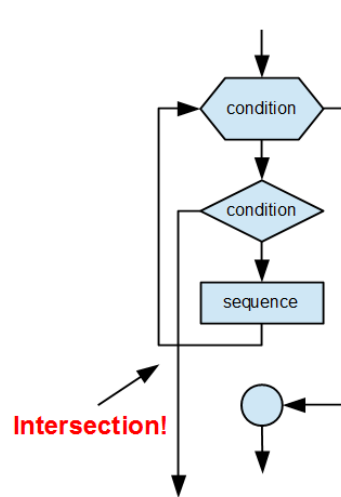
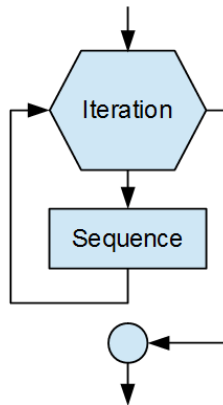
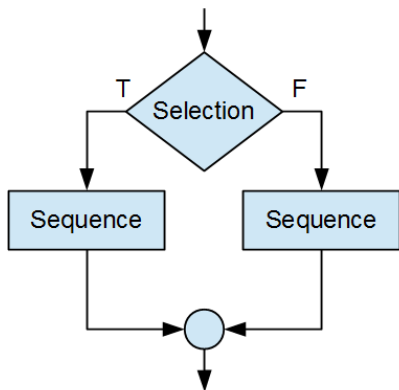
Structured Programming

Structured Programming

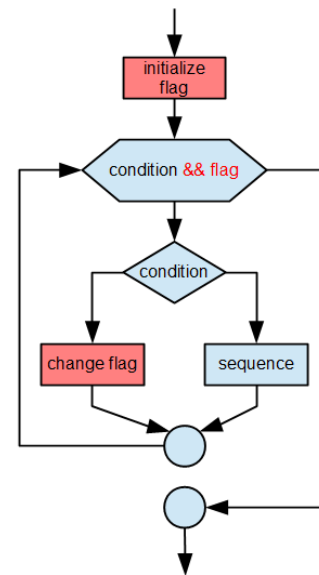
- The problem that Dijkstra recognized, early on, was that programming is hard, and that programmers don't do it very well.
- A program of any complexity contains **too many details** for a human brain to manage without help.
- During his investigation, Dijkstra discovered that certain **uses of goto** statements **prevent** modules from being decomposed recursively into smaller and smaller units.

Structured Programming (cont.)

- Dijkstra showed that all programs can be constructed from just three structures:
 - Sequence, Selection, and Iteration.



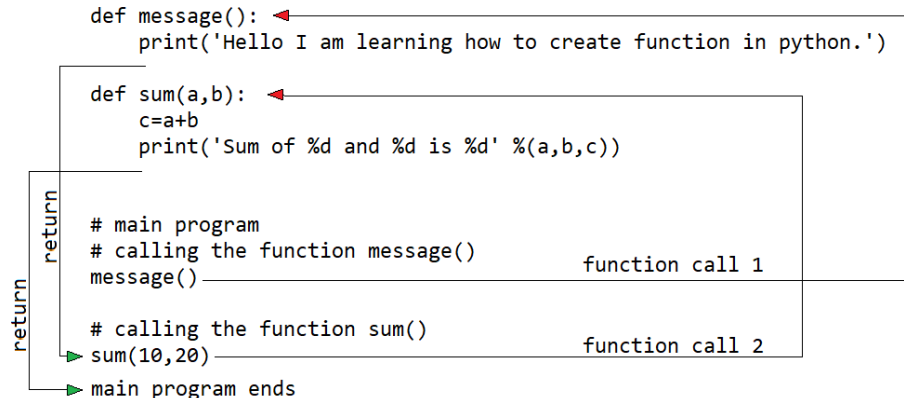
Spaghetti Code



Structured Code

Functional Decomposition

- Take a large-scale problem statement and decompose it into high level functions. Each of those functions can then be decomposed into lower-level functions, ad infinitum.
- Programmers could break down large proposed systems into **modules** and **components** that could be further broken down into tiny provable **functions**.



Tests

- Dijkstra once said, “*Testing shows the presence, not the absence, of bugs.*”
- In other words, a program can be proven incorrect by a test, but it cannot be proven correct.
- All that tests can do, after sufficient testing effort, is allow us to deem a program to be **correct enough** for our purposes.
- Software is like a **science** not like mathematics.

Object Oriented Programming

Object Oriented Programming



{OOP}

- The basis of a good architecture is the **understanding** and **application** of the principles of object-oriented design (OO).
- But, What is OO?
- One answer to this question is “The combination of data and function.”
- Another common answer to this question is “A way to model the real world.”
- Some folks fall back on three magic words to explain the nature of OO:
 - Encapsulation, Inheritance, and Polymorphism.

Encapsulation

- OO languages provide easy and effective encapsulation of **data** and **function**.
- A line can be drawn around a cohesive set of data and functions.
- Outside of that line, the data is **hidden** and only some of the functions are known.
- Indeed, the way encapsulation is partially repaired is by introducing the ***public***, ***private***, and ***protected*** keywords into the language.
- The languages that claim to provide OO have only weakened the once perfect encapsulation we enjoyed with C.

Inheritance

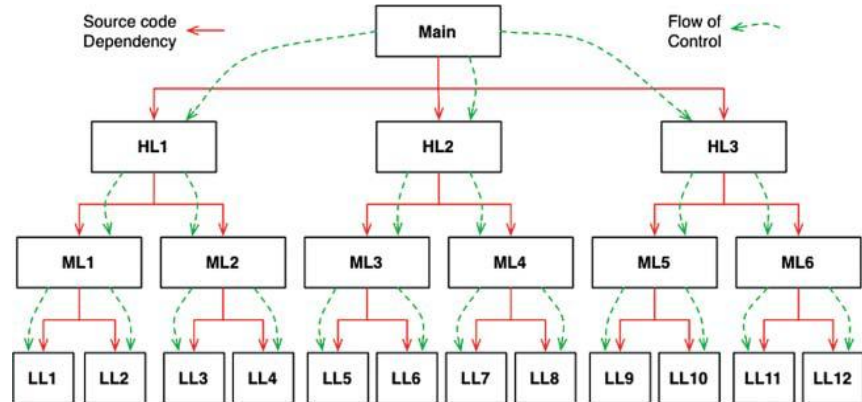
- If OO languages did not give us better encapsulation, then they certainly gave us inheritance.
- Thus, we might say that we had a kind of inheritance long before OO languages were invented.
- That statement wouldn't quite be true, though. We had a trick, but it's not nearly as convenient as true inheritance.
- Moreover, multiple inheritance is a considerably more difficult to achieve by such trickery.
- We can award no point to OO for encapsulation, and perhaps a half-point for inheritance. So far, that's not such a great score.

Polymorphism

- Programmers have been using pointers to functions to achieve polymorphic behavior since Von Neumann architectures were first implemented in the late 1940s.
- In other words, OO has provided nothing new.
- OO languages may not have given us polymorphism, but they have made it **much safer** and much **more convenient**.

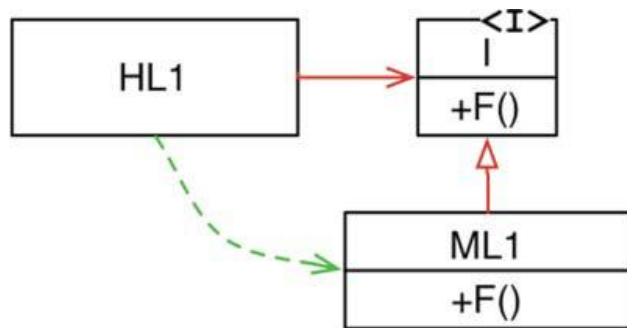
Dependency Inversion

- Imagine what software was like before a safe and convenient mechanism for polymorphism was available.
In the typical calling tree, main functions called high-level functions, which called mid-level functions, which called low-level functions.
- In that calling tree, however, source code dependencies inexorably followed the flow of control.



Dependency Inversion (cont.)

- The flow of control was dictated by the behavior of the system, and the source code dependencies were dictated by that flow of control.
- When polymorphism is brought into play, however, something very different can happen.
- At runtime, the interface doesn't exist. HL1 simply calls F() within ML1.7



Dependency inversion

Dependency Inversion (cont.)

- the source code dependency (the inheritance relationship) between *ML1* and the interface *I* points in the opposite direction compared to the flow of control.
- This is called dependency inversion, and its implications for the software architect are profound.
- The fact that OO languages provide safe and convenient polymorphism means that **any source code dependency, no matter where it is, can be inverted.**

Dependency Inversion (cont.)

- With this approach, software architects working in systems written in OO languages have **absolute control** over the direction of all **source code dependencies** in the system.
- They are not constrained to align those dependencies with the flow of control.
No matter which module does the calling and which module is called, the software architect can point the source code dependency in **either direction**.
- ***That is power!*** That is the power that OO provides. That's what OO is really all about—at least from the architect's point of view.

Dependency Inversion (cont.)

- In short, when the source code in a component changes, only that component needs to be redeployed.
*This is **independent deployability**.*
- If the modules in your system can be deployed independently, then they can be developed independently by different teams.
*That's **independent developability**.*

Conclusion

- What is OO?
- There are many opinions and many answers to this question.
- To the software architect, however, the answer is clear: OO is the ability, through the use of polymorphism, to gain absolute control over every source code dependency in the system.
- It allows the architect to create a plugin architecture, in which modules that contain high-level policies are independent of modules that contain low-level details.
- The low-level details are relegated to plugin modules that can be deployed and developed **independently** from the modules that contain high-level policies.

Functional Programming

Functional Programming

- The concepts of functional programming predate programming itself.
- This paradigm is strongly based on the λ -calculus invented by Alonzo Church in the 1930s.

Functional Programming (cont.)

- Let's investigate a simple problem:
Printing the squares of the first 25 integers.

```
public class Squint {  
    public static void main(String args[]) {  
        for (int i=0; i<25; i++)  
            System.out.println(i*i);  
    }  
}
```

- In a language like Clojure, which is a derivative of Lisp, and is functional, we might implement this same program as follows:

```
(println (take 25 (map (fn [x] (* x x)) (range))))
```

Functional Programming (cont.)

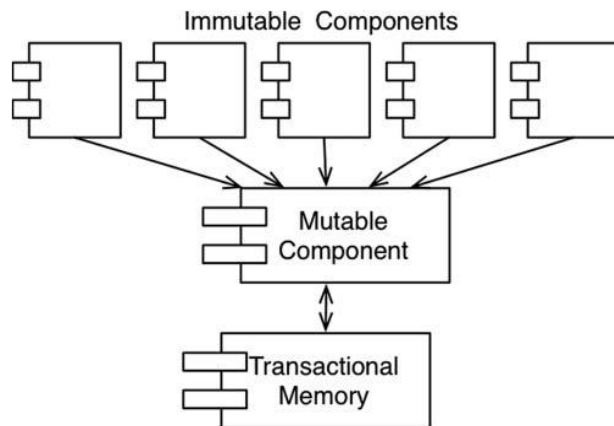
- The Java program uses a mutable variable—a variable that changes state during the execution of the program (That variable is *i*—the loop control variable).
- No such mutable variable exists in the Clojure program.
In the Clojure program, variables like *x* are initialized, but they are **never modified**.
- This leads us to a surprising statement:
Variables in functional languages do not vary.

Immutability and Architecture

- Why is this point important as an architectural consideration? Why would an architect be concerned with the mutability of variables?
- The answer is absurdly simple:
All **race conditions**, **deadlock conditions**, and **concurrent update problems** are due to mutable variables.
- You cannot have a race condition or a concurrent update problem if no variable is ever updated.
You cannot have deadlocks without mutable locks.
- As an architect, you should be very interested in issues of **concurrency**.

Segregation of Mutability

- Segregate the application, or the services within the application, into mutable and immutable components.
- The immutable components perform their tasks in a purely functional way, without using any mutable variables.



Mutating state and transactional memory

Segregation of Mutability (cont.)

- The immutable components communicate with one or more other components that are not purely functional, and allow for the state of variables to be mutated.
- Architects would be wise to push as much processing as possible into the **immutable components**, and to drive as much code as possible **out of** those components that must allow mutation.

Conclusion

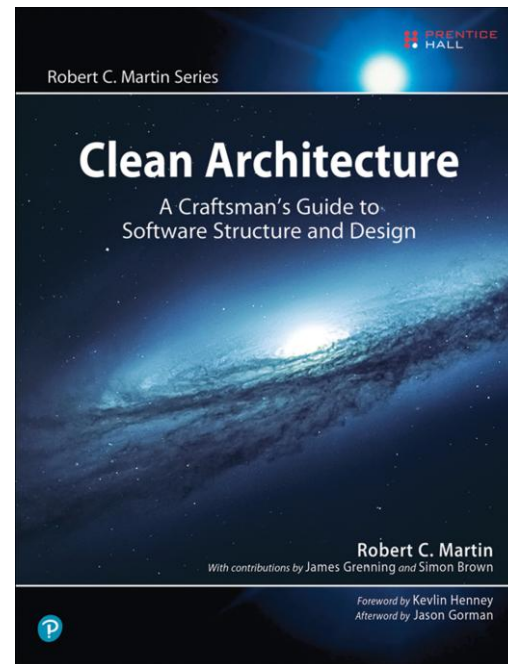
- Structured programming is discipline imposed upon **direct transfer of control**.
- Object-oriented programming is discipline imposed upon **indirect transfer of control**.
- Functional programming is discipline imposed upon **variable assignment**.
- Each of these three paradigms has taken something away from us. Each restricts some aspect of the way we write code. None of them has added to our power or our capabilities.

Conclusion (cont.)

- What we have learned over the last half-century is **what not to do**.
- Software is **not** a rapidly advancing technology.
The rules of software are the **same** today as they were in 1946, when Alan Turing wrote the very first code that would execute in an electronic computer.
- The **tools** have changed, and the **hardware** has changed, but the essence of **software** remains the same.
- Software—the stuff of computer programs—is composed of sequence, selection, iteration, and indirection. Nothing more. Nothing less.

Resources

- Clean Architecture (Robert C. Martin): Chapters 1, 2, 3, and 4.



Q&A
